

Análisis Numérico

Trabajo Práctico 3

Segundo cuatrimestre 2026

Instrucciones:

- Fecha de presentación: 20/07/26.
- Los grupos se conforman de 3 o 4 personas.
- Utilice Python como lenguaje de programación.
- Elabore un informe lo más detallado posible, mencionando los problemas con los que se encontró intentando obtener las respuestas a las consignas.
- Subir al Campus Virtual en un archivo comprimido único: **el informe en formato .pdf** y cualquier otro archivo que considere útil, como códigos u otros.
- Elaborar un video de no más de 3 minutos de duración sobre los aspectos más importantes del proceso y las conclusiones del trabajo. Subir el video al equipo de TEAMS.

1. Algoritmos para encontrar raíces en funciones no lineales

El objetivo de este TP3 es diseñar e implementar en Python un algoritmo híbrido de búsqueda de raíces de funciones no lineales, capaz de encontrar las soluciones para expresiones complejas (notablemente no lineales, oscilatorias, con raíces múltiples o asíntotas). Como parte de la consigna, no se permite el uso de funciones nativas de optimización del lenguaje de programación, ni paquetes externos como **numpy**, **scipy** u otros. Pueden emplearse las funciones matemáticas incluidas en el módulo **math**. El trabajo limitará su alcance al campo de los números reales.

Cada grupo deberá implementar un método numérico iterativo que combine metodologías clásicas con eventuales métodos personalizados, el que deberá resolver exitosamente el cálculo de la raíz (convergencia) en cada una de las funciones de prueba que se listan en la Tabla 1. Asimismo, la tabla indica el intervalo que contiene a la raíz (única en dicho rango), y debe ser interpretado como el *dominio restringido* de la función.

El algoritmo híbrido propuesto por el grupo será evaluado bajo un sistema de *benchmark*, que pondera el costo computacional asociado a su ejecución para hallar las raíces de las funciones de la Tabla 1. El sistema utiliza un esquema que suma puntos de penalización de acuerdo con el tipo y cantidad de cálculos realizados (no depende del tiempo de cómputo). Finalmente, se aplica una conversión del puntaje final a una escala numérica de 0 a 10, correspondiente a la calificación global del método. Luego, cuanto menor sea el puntaje final acumulado por el método (menor afección por penalizaciones), mayor será la calificación global. Para facilidad de los estudiantes, el archivo **Funciones_Prueba.txt** contiene a las funciones de la Tabla 1 en sintaxis Python. No obstante, los estudiantes no tendrán acceso al software de benchmark que utilizará el equipo docente para la evaluación. En su lugar, deberán considerar los detalles del sistema de penalizaciones, según se explica en los apartados 2.2 y 2.3.

Tabla 1: Funciones de prueba para evaluar el desempeño del método numérico.

#	Función	Intervalo que contiene a la raíz
1	$f(x) = (x - 1,5)^4 \cdot \ln(x^2 + 1)$	[1,0; 2,0]
2	$f(x) = (x - 0,75)^7$	[0,0; 1,0]
3	$f(x) = x^{11} - 10^{-4}$	[0,0; 1,0]
4	$f(x) = \sin\left(\frac{1}{x + 0,1}\right) - 0,1$	[0,1; 0,4]
5	$f(x) = \tan x - x - 1$	[1,0; 1,5]
6	$f(x) = \sinh(x^5) - 100$	[0,5; 2,0]
7	$f(x) = e^{-10x} - 10^{-5}$	[0,0; 2,0]
8	$f(x) = \ln(x - 2 + 10^{-15}) + 5$	[2,0; 3,0]
9	$f(x) = (x^2 - 1,2)^2 - 0,1$	[1,0; 1,5]
10	$f(x) = \left[\prod_{k=1}^5 (x - k)\right] - 0,01$	[4,5; 5,2]
11	$f(x) = e^x - \cos x - 10$	[2,0; 3,5]
12	$f(x) = \sqrt[3]{x - 2}$	[1,0; 4,0]
13	$f(x) = \frac{x}{x^2 + 1} - 0,45$	[0,4; 0,8]
14	$f(x) = x^5 - 5x^3 + 4x - 0,5$	[1,8; 2,3]
15	$f(x) = e^{-x} \cdot \sin(2\pi x) - 10^{-3}$	[0,2; 0,6]
16	$f(x) = \arctan(x - 1250,45) - 0,1$	[-1000,0; 1000,0]
17	$f(x) = \frac{x}{ x + 1} - 0,9999$	[-200000,0; 200000,0]

2. Criterios para el diseño del algoritmo

El grupo deberá generar un archivo **metodo.py** con el método híbrido propuesto, respetando estrictamente la siguiente estructura. El algoritmo deberá definirse en la función **encontrar_raiz()**. En ésta, podrán ejecutarse varios métodos numéricos en secuencia (concatenados), con la restricción de que cada uno, individualmente, no compute más de 150 iteraciones.

```

1 # No se permite el uso de librerías externas (numpy, scipy, etc.).
2 # Sólo se permite el uso del módulo estándar 'math'.
3 # Iteraciones máximas del algoritmo: 150
4
5 def encontrar_raiz(f, a, b, tol):
6     """
7     Argumentos de entrada:
8     f : Función a evaluar. Se debe invocar especificando el "tipo" de llamada:
9         - Para evaluar la función de forma estándar: f(x, tipo="comun")
10        - Para evaluar puntos destinados al cálculo de derivadas: f(x, tipo="derivada")
11     a, b: Flotantes. Extremos del intervalo inicial (b > a).
12     tol : Flotante>0. Tolerancia de convergencia.
13         El algoritmo DEBE detenerse estrictamente cuando | x_k - x_{k-1} | <= tol.
14         Para la evaluación, el equipo docente utilizará: tol = 1e-12

```

```

15
16 Retorno:
17     Devuelve únicamente un número flotante con la estimación de la raíz.
18     """
19     # Ejemplo de llamadas válidas para la telemetría del peaje (sistema de benchmark):
20     # fx = f(x, tipo="comun")
21     # fx_der = f(x + dx, tipo="derivada")
22
23     max_iter = 150
24
25     # Completar código...
26
27     return raiz_estimada

```

Prestar especial atención a la forma de invocar a la función matemática $f(x)$. Cada vez que el método evalúe $f(x)$, deberá ejecutarse de la siguiente manera:

```

1 y = f(x, tipo="comun")

```

Por otra parte, el algoritmo a desarrollar no puede utilizar derivadas exactas (como en *Newton-Raphson clásico*). En su lugar, si una derivada es requerida, puede aproximarse como un cociente incremental, de la siguiente manera (elegir convenientemente el valor de **delta_x**):

```

1 delta_x = 0.1 # Esta cantidad puede ser diferente
2 y1 = f(x, tipo="comun")
3 y2 = f(x + delta_x, tipo="derivada") # Señala que se calculó una derivada
4
5 f_derivada = (y2 - y1) / delta_x

```

El uso de la sintaxis adecuada para la función **f()**, de acuerdo con los tipos “comun” y “derivada”, permitirá ejecutar el benchmark aplicando correctamente las penalizaciones.

Por otro lado, se destaca que, desde la cátedra, se proveen los archivos **biseccion.py** y **NR.py**, con implementaciones de los métodos de *Bisección* y *Newton-Raphson* (con derivada numérica aproximada), respectivamente. Cada grupo podrá utilizar libremente estas funciones, si así lo dispone, de la siguiente forma:

```

1 import math
2 import biseccion
3 import NR
4
5 f = lambda x: ((x - 1.5) ** 4) * math.log(x ** 2 + 1) # ejemplo con la función de prueba nro. 1
6 tol = 1e-12 # tolerancia de convergencia
7
8 raiz_biseccion = biseccion.encontrar_raiz(f, 1, 2, tol)
9 raiz_NR = NR.encontrar_raiz(f, 1, 2, tol)

```

Los métodos dispuestos en los módulos **biseccion** y **NR** no pueden modificarse. Si el grupo opta por utilizarlos, deberá hacer uso de estas implementaciones, prohibiéndose versiones modificadas. Advertir que **NR** define al punto inicial como el valor medio del intervalo $((a+b)/2)$, y que ambos métodos definen la convergencia para una diferencia entre 2 iteraciones sucesivas (en valor absoluto) menor o igual que **tol**.

$$|x_k - x_{k-1}| \leq \text{tol} \quad (\text{condición de detención por convergencia en la } k\text{-ésima iteración}) \quad (1)$$

IMPORTANTE:

Aún en implementaciones de métodos abiertos, la función **encontrar_raiz()** debe respetar la signatura:

def encontrar_raiz(f, a: float, b: float, tol: float) -> float:

donde **a** y **b** serán sustituidos por el software de benchmark con los extremos del intervalo indicado en la Tabla 1. Reducir estos datos a un punto inicial en métodos abiertos, implicará un criterio personalizado del grupo para la implementación del algoritmo.

2.1. Métodos permitidos

Para construir el algoritmo maestro, se permite combinar e hibridar libremente los siguientes métodos:

- Bisección (archivo **biseccion.py**).

- Método de Newton-Raphson con aproximación de derivada numérica (archivo **NR.py**).
- Algoritmo personalizado a elección del grupo.

Se sugiere comenzar la búsqueda de la raíz de forma segura (acotando la solución) y, a medida que el intervalo se reduzca, mutar inteligentemente a métodos abiertos de orden superior para acelerar la convergencia. Es importante, también, detectar si un método rápido calcula un resultado absurdo (por ejemplo, fuera del intervalo de confinamiento), descartar ese paso erróneo y aplicar una reestabilización geométrica inmediata.

2.2. Sistema de puntuación y penalizaciones

La eficiencia no se medirá en tiempo de ejecución (el cual fluctúa según el hardware), sino en **puntos de costo acumulados** a lo largo de una suite de 17 funciones de prueba (según la Tabla 1). Cada interacción con las funciones sumará puntos (penalización) bajo las siguientes reglas económicas:

- Llamada Común:** Suma **1 punto** por cada evaluación de $f(x)$ invocada como **f(x, tipo="comun")**.
- Llamada de Derivada:** Suma **2 puntos** por cada evaluación invocada como **f(x, tipo="derivada")**.
- Impuesto a la Ráfaga Monótona:** Penaliza la insistencia para evitar el uso de algoritmos planos o puros (como bisección pura, o secante pura). Cobra **4 puntos** en lugar de 1 por cada llamada **tipo="comun"** a partir de la quinta, siempre que el algoritmo realice más de **4 llamadas comunes consecutivas**. Se aplica la misma penalización al encadenar más de 4 llamadas de derivada consecutivas (**tipo="derivada"**).
- Reinicio Cruzado:** Reinicia inmediatamente a cero la ráfaga de llamadas **tipo="comun"** cuando el algoritmo ejecute una llamada de **tipo="derivada"**. De igual forma, corta y reinicia la racha de derivadas (**tipo="derivada"**) mediante una llamada de **tipo="comun"**.

Sugerencia: Almacenar resultados parciales en variables y evitar recalcular imágenes en abscisas ya conocidas, con la finalidad de reducir la cantidad de puntos evaluados.

2.3. Multas del sistema

El sistema de puntaje complementa las penalizaciones por evaluación de la función y su derivada, adicionando multas de acuerdo con el resultado obtenido (estimación de la raíz), en relación al error cometido (diferencia con el resultado exacto). Este tipo de multa se aplica para cada una de las 17 ejecuciones del algoritmo ensayado, utilizando todas las funciones de prueba. En este esquema, se aplican penalizaciones nulas o severas de acuerdo con los siguientes comportamientos posibles:

- **Aproximación Exitosa:** El algoritmo se detiene tras alcanzar la condición de la Ec. (1), cumpliendo con **tol** $\leq 10^{-12}$ y logrando un error absoluto respecto a la raíz exacta menor o igual a 10^{-7} (error $\leq 10^{-7}$). En este caso, no se consideran puntos adicionales (**+0 puntos de penalización**).
- **Raíz Incorrecta (Falso Positivo):** El programa se detiene prematuramente o devuelve un valor cuya distancia a la raíz esperada es mayor a 10^{-7} , considerándose un error de posición (multa: **+1000 puntos de penalización**).
- **Colapso del Sistema:** Se arroja una excepción de Python (como división por cero no controlada, o desbordamiento flotante), quedando bloqueado en un bucle infinito o excediendo el límite estricto de 150 iteraciones sin converger (multa severa: **+1500 puntos de penalización**).

El diseño del mejor balance entre intervalos acotados y la aceleración por interpolación/derivadas, logrará conseguir el puntaje más bajo y, por consecuencia, la nota máxima del TP.

3. Acerca del informe

La evaluación no se limitará a la presentación del código fuente final. Se otorgará especial relevancia a la calidad del análisis realizado, a la justificación de las decisiones de diseño adoptadas y a la documentación integral del proceso de desarrollo.

El informe deberá incluir una descripción detallada de la estrategia heurística propuesta para resolver el problema del peaje impositivo, explicitando los criterios utilizados para la selección y combinación de métodos numéricos. Asimismo, deberán justificarse matemáticamente las condiciones bajo las cuales el algoritmo

modifica su estrategia de resolución (por ejemplo, la transición desde un método cerrado, como *Bisección*, hacia un método abierto).

Se recomienda la inclusión de diagramas de flujo u otras representaciones esquemáticas que permitan describir con claridad la lógica del algoritmo y, en particular, el funcionamiento de su bucle principal de control.

Además, se deberá documentar el proceso de experimentación y refinamiento que condujo al diseño final. En este sentido, resulta importante registrar las distintas versiones preliminares del algoritmo, describir las modificaciones introducidas en cada etapa y analizar la evolución de las métricas de desempeño obtenidas, prestando especial atención al efecto de los criterios de parada y de conmutación entre métodos.

El informe deberá incorporar tablas comparativas que permitan evaluar el rendimiento de las arquitecturas propuestas frente a los métodos puros provistos por la cátedra. Entre los indicadores a considerar se incluyen, entre otros, la cantidad de evaluaciones de la función, la cantidad de evaluaciones de derivadas y el puntaje de costo final obtenido para cada función de prueba.

Finalmente, se espera una reflexión crítica sobre el compromiso entre robustez y velocidad de convergencia, destacando las lecciones aprendidas respecto del diseño de software numérico bajo restricciones de costo computacional.